Intro to Data Science - HW 6

Copyright Jeffrey Stanton 2025, Jeffrey Saltz, Christopher Dunham, and Jasmina Tacheva

```
# Enter your name here: Sofia Quintero
```

USE OF ARTIFICIAL INTELLIGENCE: While you are allowed to use generative AI as a learning tool for this assignment (e.g., to help with brainstorming, to see how to code a certain concept, to help identify flaws in reasoning, or to spot confusing or underdeveloped code segments), it must not be used to input homework questions, problems, or instructions directly, and you may not copy and paste any AI-generated content into your work. By submitting this assignment, you confirm that all the work is your own.

Attribution statement: (choose only one and delete the rest)

```
# 1. I did this homework by myself, with help from the book and the professor.
```

Data visualization is important because many people can understand data better when it is presented graphically. As a data scientist, you will have to present complex data to decision makers in a way they can understand. Based on lecture and your experience with Excel and other tools, you know there are numerous data visualization types (e.g., line charts). How many of them can you name?

The most powerful tool for data visualization in R is called ggplot. Written by computer/data scientist Hadley Wickham, this graphics grammar tool builds visualizations in layers. This method provides immense flexibility, but takes a bit of practice to master.

Step 1: Make a copy of the data

A. Read the **Syracuse Code Violations** data from this URL: "https://data-science-intro.s3.us-east-2.amazonaws.com/Code_Violations.csv" into a dataframe called **violations_df**.

```
library("tidyverse")
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
violations_df <- read_csv("https://data-science-intro.s3.us-east-2.amazonaws.com/Code_Violations.csv")
```

```
## Rows: 132250 Columns: 26
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (17): violation_id, complaint_address, SBL, violation, violation_date, c...
## dbl  (9): X, Y, number_violations, complaint_zip, zip_median_income, inspect...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

B. Your dataframe, **violations_df**, contains a list of code violations recorded repeatedly over time (2017-2024). Identify the variable that captures the date a violation has occurred and inspect it.

```r
head(violations_df)
```

```
## # A tibble: 6 x 26
##       X     Y violation_id number_violations complaint_address     complaint_zip
##   <dbl> <dbl> <chr>                    <dbl> <chr>                         <dbl>
## 1 -76.1  43.0 2023-19961                   1 350 Montgomery St & ~         13202
## 2 -76.1  43.0 2023-07624                   1 350 Montgomery St & ~         13202
## 3 -76.1  43.0 2023-07623                   1 350 Montgomery St & ~         13202
## 4 -76.1  43.0 2023-07622                   1 350 Montgomery St & ~         13202
## 5 -76.1  43.0 84997                        1 350 Montgomery St & ~         13202
## 6 -76.1  43.0 84998                        1 350 Montgomery St & ~         13202
## # i 20 more variables: zip_median_income <dbl>, SBL <chr>, violation <chr>,
## #   violation_date <chr>, comply_by_date <chr>, status_type_name <chr>,
## #   complaint_number <chr>, complaint_type_name <chr>, open_date <chr>,
## #   owner_name <chr>, inspector_id <dbl>, Neighborhood <chr>, Vacant <chr>,
## #   owner_address <chr>, owner_city <chr>, owner_state <chr>,
## #   owner_zip_code <chr>, Latitude <dbl>, Longitude <dbl>, ObjectId <dbl>
```

What is the level of detail captured in this variable (weekly, monthly, etc.)? Answer in a comment.

```r
# violation_date indicates when the violation has occurred it captures day,
# month, and year it occurred as well as time to the millisecond
```

C. What is the data type of this variable? Explain in a comment and show the command you used to find out.

```r
typeof(violations_df$violation_date)
```

```
## [1] "character"
```

```r
# violation_date is a character
```

D. To properly display the time values in our plots as dates, we must convert the time variable to date format using the **as.Date()** function. Run the code below, comment on how it works, and double-check the variable's data type to ensure it is no longer coded as text.

```r
violations_df$violation_date <- as.Date(violations_df$violation_date)
# Comment: as.Date converts character data to a date object class
```

```r
typeof(violations_df$violation_date)
```

```
## [1] "double"
```

Step 2: Dealing with missing data and subsetting

A. When you first start working with a dataset, it is always a good idea to look for missing values. Check the variables **number_violations** and **zip_median_income** for missing values using the **is.na()** function. Are there any?

```r
nrow(subset(violations_df, is.na(violations_df$number_violations)))
```

```
## [1] 0
```

```r
nrow(subset(violations_df, is.na(violations_df$zip_median_income)))
```

```
## [1] 22
```

B. **imputeTS** is a R package that is specifically designed to 'repair' missing values in time series data. We'll use this instead of the simpler **mean substitution** method because it's more accurate. If there are missing values in the two variables, use the na_interpolation() function from the imputeTS package to interpolate the missing values. Make sure to first sort the dataframe by violation_date and then by complaint_zip before applying the function.

```r
install.packages("imputeTS", repos = "http://cran.us.r-project.org")
```

```
##
## The downloaded binary packages are in
##  /var/folders/rm/s4vv52kd55g34xzsxxqn9p2c0000gn/T//RtmpbyDYQw/downloaded_packages
```

```r
library("imputeTS")
```

```
## Registered S3 method overwritten by 'quantmod':
##   method            from
##   as.zoo.data.frame zoo
```

```r
violations_df[order(violations_df$complaint_zip, decreasing = TRUE), ]
```

```
## # A tibble: 132,250 x 26
##        X     Y violation_id number_violations complaint_address    complaint_zip
##    <dbl> <dbl> <chr>                    <dbl> <chr>                        <dbl>
##  1 -76.1  43.1 2024-04801                   1 117 Bruce St                 13224
##  2 -76.1  43.1 2024-04800                   1 117 Bruce St                 13224
##  3 -76.1  43.1 2024-04799                   1 117 Bruce St                 13224
##  4 -76.1  43.1 2023-06590                   1 117 Bruce St                 13224
##  5 -76.1  43.1 2022-16659                   1 117 Bruce St                 13224
##  6 -76.1  43.0 2023-13398                   1 2116-24 Fayette St ~         13224
##  7 -76.1  43.0 2023-08224                   1 2116-24 Fayette St ~         13224
```

```
##  8 -76.1  43.0 2021-07292                  1 2116-24 Fayette St ~         13224
##  9 -76.1  43.0 69213                       1 2116-24 Fayette St ~         13224
## 10 -76.1  43.1 2023-06596                  1 2030 Erie Blvd E             13224
## # i 132,240 more rows
## # i 20 more variables: zip_median_income <dbl>, SBL <chr>, violation <chr>,
## #   violation_date <date>, comply_by_date <chr>, status_type_name <chr>,
## #   complaint_number <chr>, complaint_type_name <chr>, open_date <chr>,
## #   owner_name <chr>, inspector_id <dbl>, Neighborhood <chr>, Vacant <chr>,
## #   owner_address <chr>, owner_city <chr>, owner_state <chr>,
## #   owner_zip_code <chr>, Latitude <dbl>, Longitude <dbl>, ObjectId <dbl>
```

```
violations_df$zip_median_income <- na_interpolation(violations_df$zip_median_income)
```

C. To ensure that there is no more missing data, run the code from Step 2A again:

```
nrow(subset(violations_df, is.na(violations_df$zip_median_income)))
```

```
## [1] 0
```

D. Create a new dataframe containing only the data for the "Northside" neighborhood using subsetting.

Hint: To avoid including rows with missing values, add this condition to your subsetting code: & !is.na(violations_df$Neighborhood$).Forexample, filteringforthe"Southside"neighborhoodwouldlooklikethis$: [$violations_df$Neighborhood == "Southside" & !is.na(violations_df$Neighborhood),]

```
northside_df <- subset(violations_df, violations_df$Neighborhood == "Northside" &
    !is.na(violations_df$Neighborhood))
```

E. Using the same logic, create a separate dataframe for the following neighborhood: Westcott

```
westcott_df <- subset(violations_df, violations_df$Neighborhood == "Westcott" & !is.na(violations_df$Nei
```

F. Using the same logic, create a separate dataframe for the following neighborhood: Downtown

```
downtown_df <- subset(violations_df, violations_df$Neighborhood == "Downtown" & !is.na(violations_df$Nei
```

G. Finally, using the same logic, create a separate dataframe for the University area which combines two neighborhoods: University Neighborhood and University Hill.

```
university_df <- subset(violations_df, grepl("University", violations_df$Neighborhood) &
    !is.na(violations_df$Neighborhood))
```

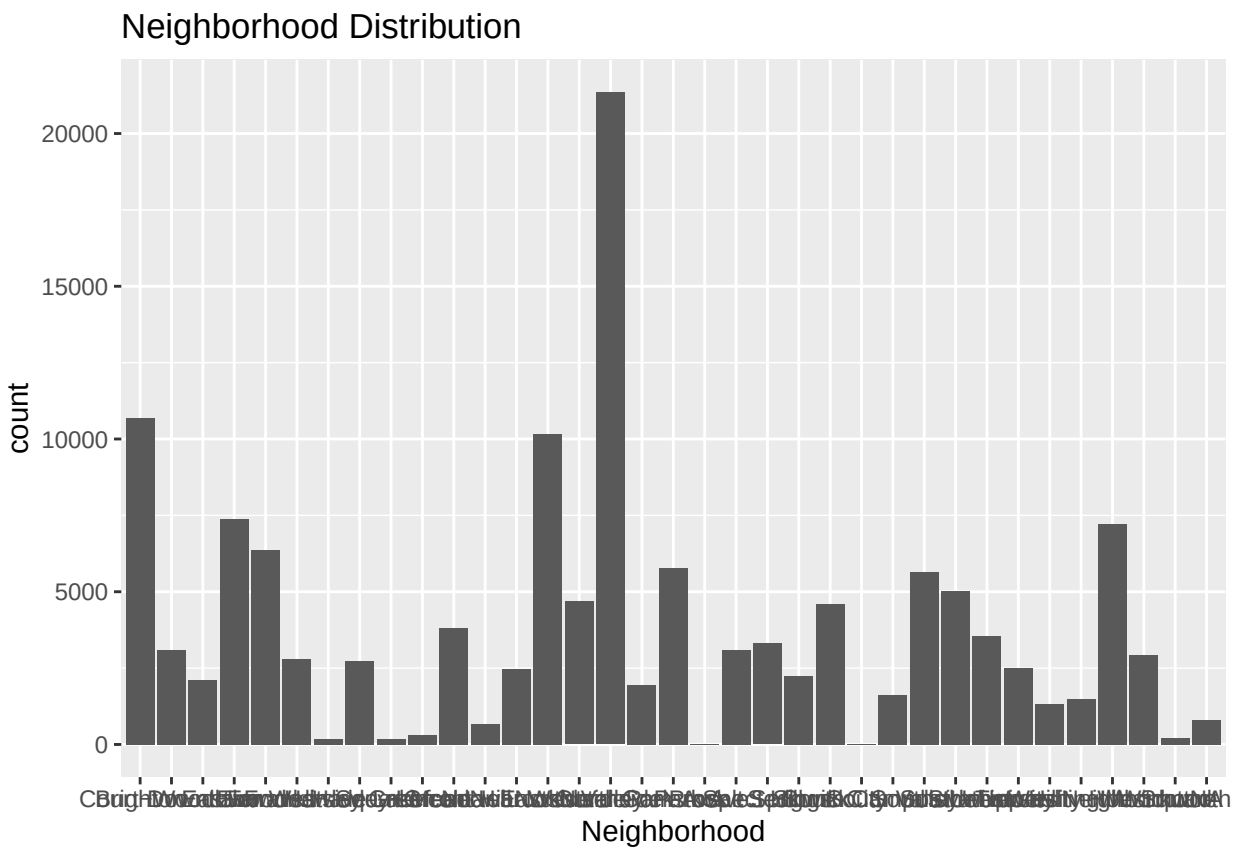
Step 3: Use ggplot to explore the distribution of the data

Don t forget to install and library the ggplot2 package. Then: A. Create a bar chart for **Neighborhood** in the **violations df** dataframe. Make sure to add a title. In a comment, briefly interpret the histogram.

```r
install.packages("ggplot2", repos = "http://cran.us.r-project.org")
```

```
##
## The downloaded binary packages are in
##  /var/folders/rm/s4vv52kd55g34xzsxxqn9p2c0000gn/T//RtmpbyDYQw/downloaded_packages
```

```r
library("ggplot2")
neighborhoodPlot <- ggplot(violations_df, aes(x = Neighborhood)) + geom_bar() + ggtitle("Neighborhood D
neighborhoodPlot
```




```r
# Some neighborhoods have little to no violations while others range up to
# 10000 Due to the amount of neighborhoods, the x-axis becomes hard to read and
# the outlier neighborhood is hard to identify, but it can be read that it has
# over 20000 violations
```

```r
table(violations_df$Neighborhood)
```

```
##
##                Brighton          Court-Woodlawn                Downtown
##                   10681                    3107                    2107
##                Eastwood                 Elmwood            Far Westside
##                    7368                    6356                    2811
##          Franklin Square            Hawley Green            Hawley-Green
##                     188                    2741                     169
```

```
##               Lakefront            Lincoln Hill             Meadowbrook
##                     300                    3798                     672
##           Near Eastside           Near Westside            North Valley
##                    2462                   10172                    4680
##               Northside          Outer Comstock                Park Ave
##                   21343                    1936                    5779
##               Park Ave.           Prospect Hill            Salt Springs
##                       3                    3104                    3305
##                Sedgwick              Skunk City            South Campus
##                    2232                    4603                      16
##            South Valley               Southside               Southwest
##                    1616                    5651                    5013
##              Strathmore               Tipp Hill         University Hill
##                    3547                    2517                    1314
## University Neighborhood       Washington Square                Westcott
##                    1491                    7231                    2917
##               Winkworth
##                     216
```

```
# Northside is the outlier with 21343 violations
```

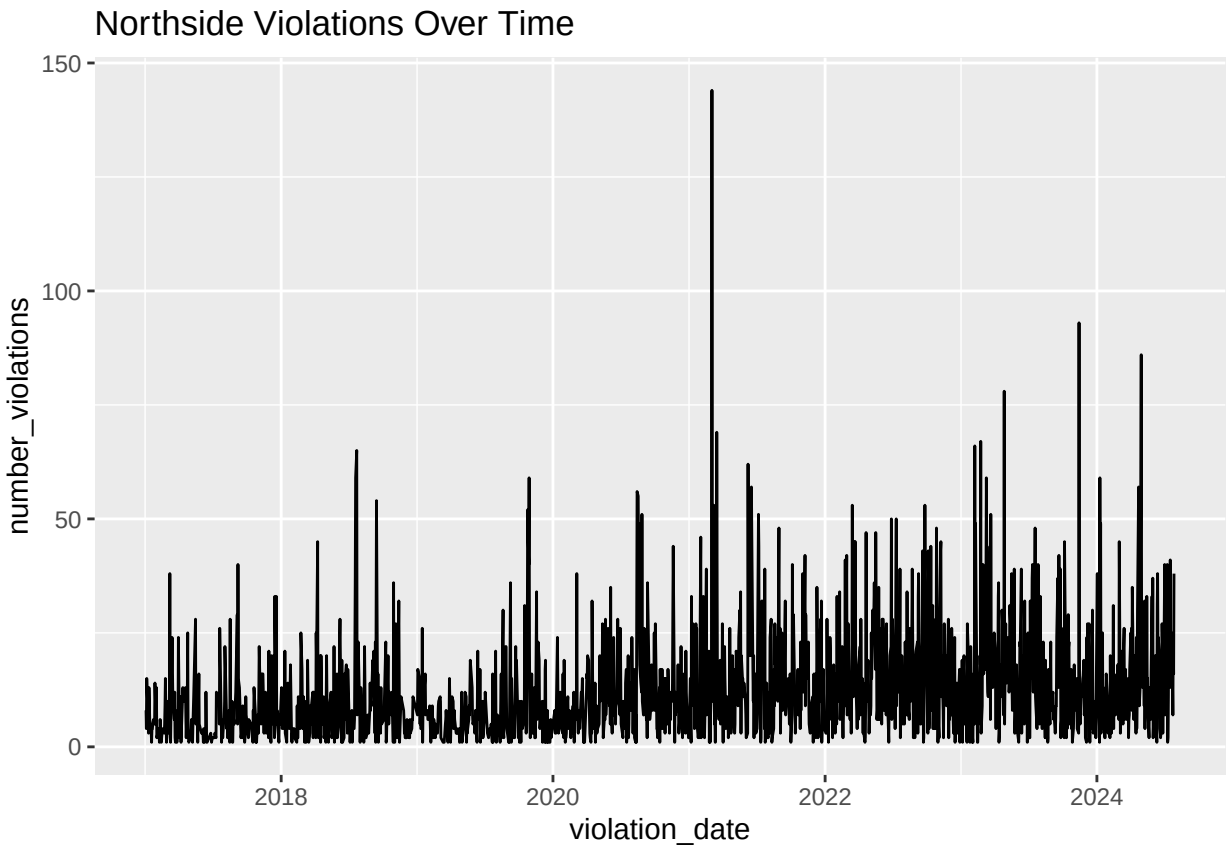
Step 4: Explore how the data changes over time

A. The data were collected over nearly 7 years. A line chart can be used to track changes over time. First, however, we need to aggregate the number of violations by date using the aggregate() function. Here is an example of how it works, applied to the overall data stored in violations_df. Use the same logic to obtain date aggregates for the four neighborhood dataframes:

```
violations_by_date <- aggregate(number_violations ~ violation_date, data = violations_df,
    sum)
northside_by_date <- aggregate(number_violations ~ violation_date, data = northside_df,
    sum)
westcott_by_date <- aggregate(number_violations ~ violation_date, data = westcott_df,
    sum)
downtown_by_date <- aggregate(number_violations ~ violation_date, data = downtown_df,
    sum)
university_by_date <- aggregate(number_violations ~ violation_date, data = university_df,
    sum)
```

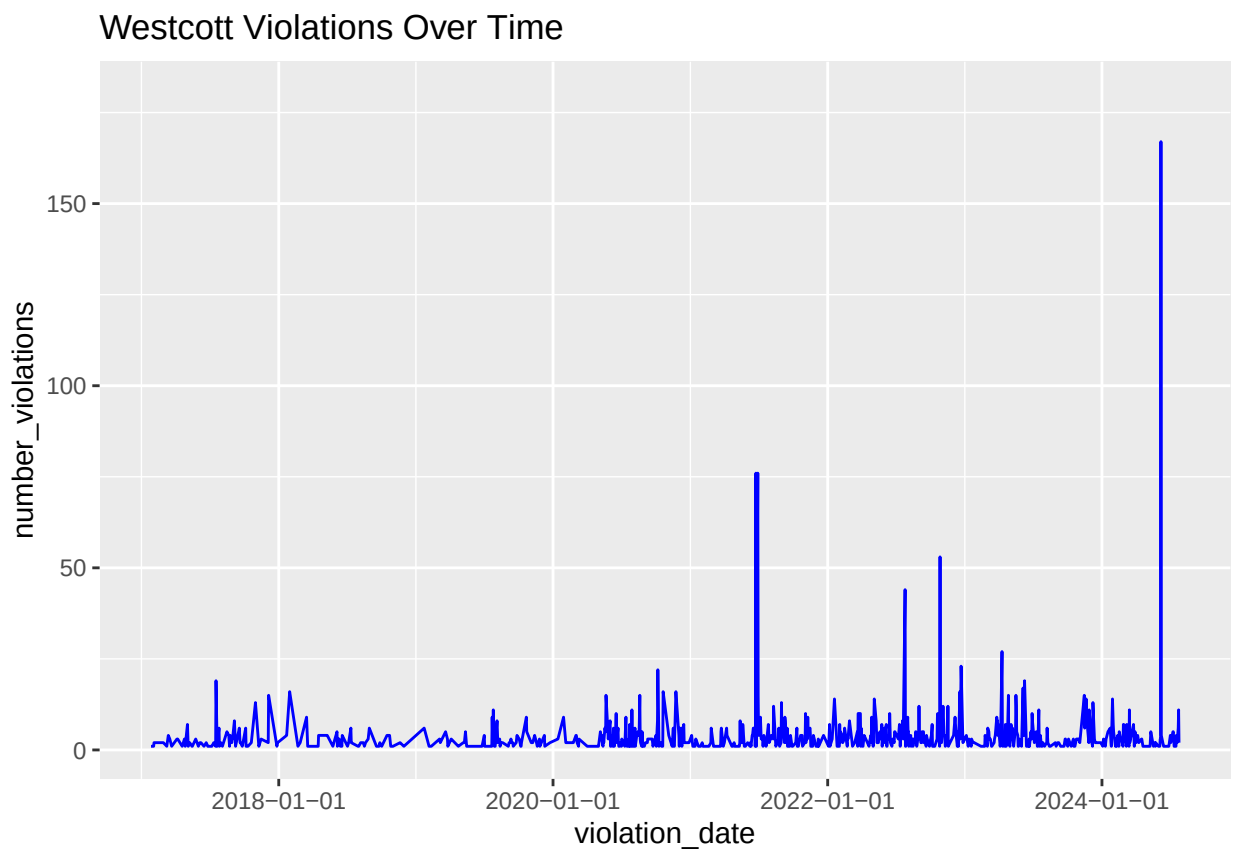
Once you are done with the previous step, we can start creating line charts to visualize the pattern of violations in each neighborhood over time. Let's focus on the **Northside df** subset first: Set the violation date variable on the X-axis and number_violations on the Y-axis of a **line chart**.

```
northsidePlot <- ggplot(northside_by_date, aes(x = violation_date)) + geom_line(aes(y = number_violatio
    ggtitle("Northside Violations Over Time")
northsidePlot
```

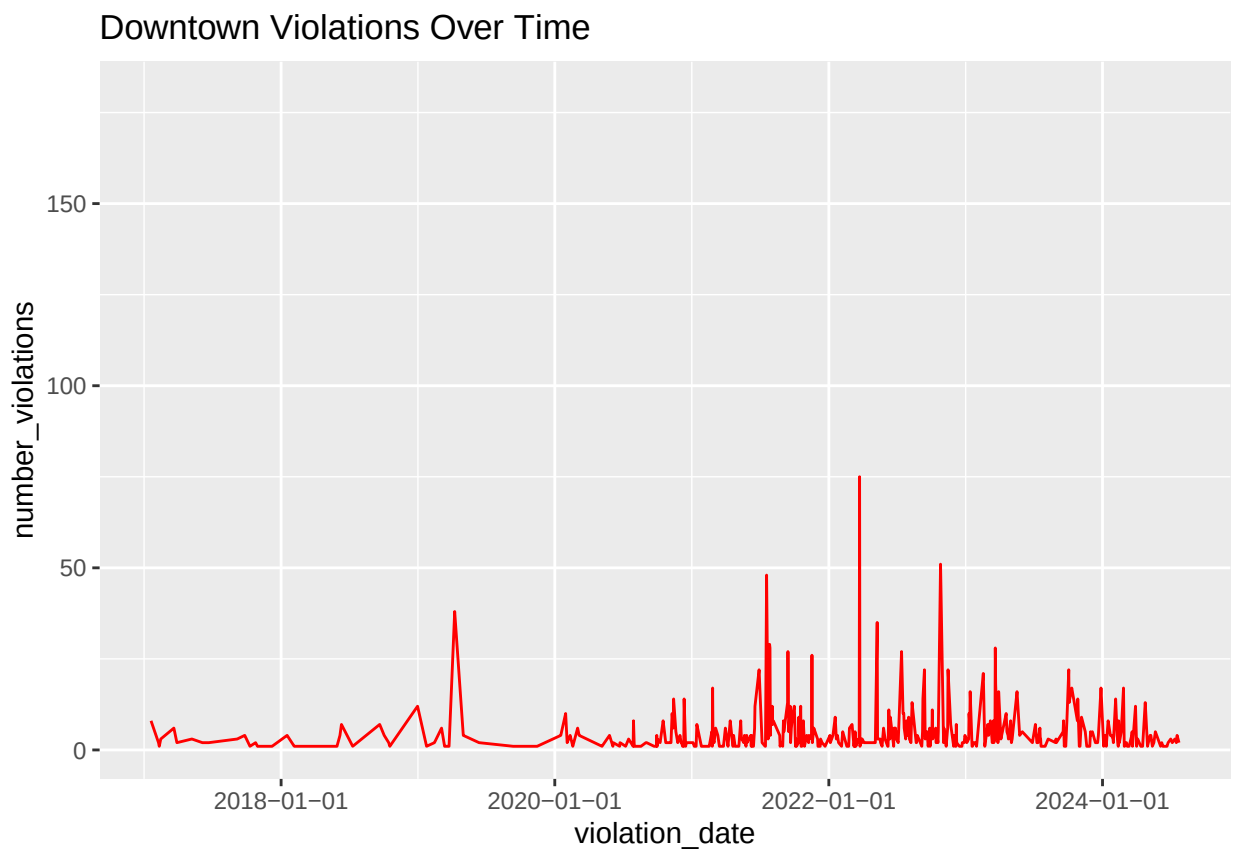


B. Next, generate similar graphs for each of the remaining three subsets. Use a different color for each line plot. For full credit, ensure that the x-axis is displayed in a more granular format, such as quarterly intervals, and set the y-axis to extend up to at least 180, e.g. by using the ylim() function. This will allow for effective comparison of patterns across the different plots.

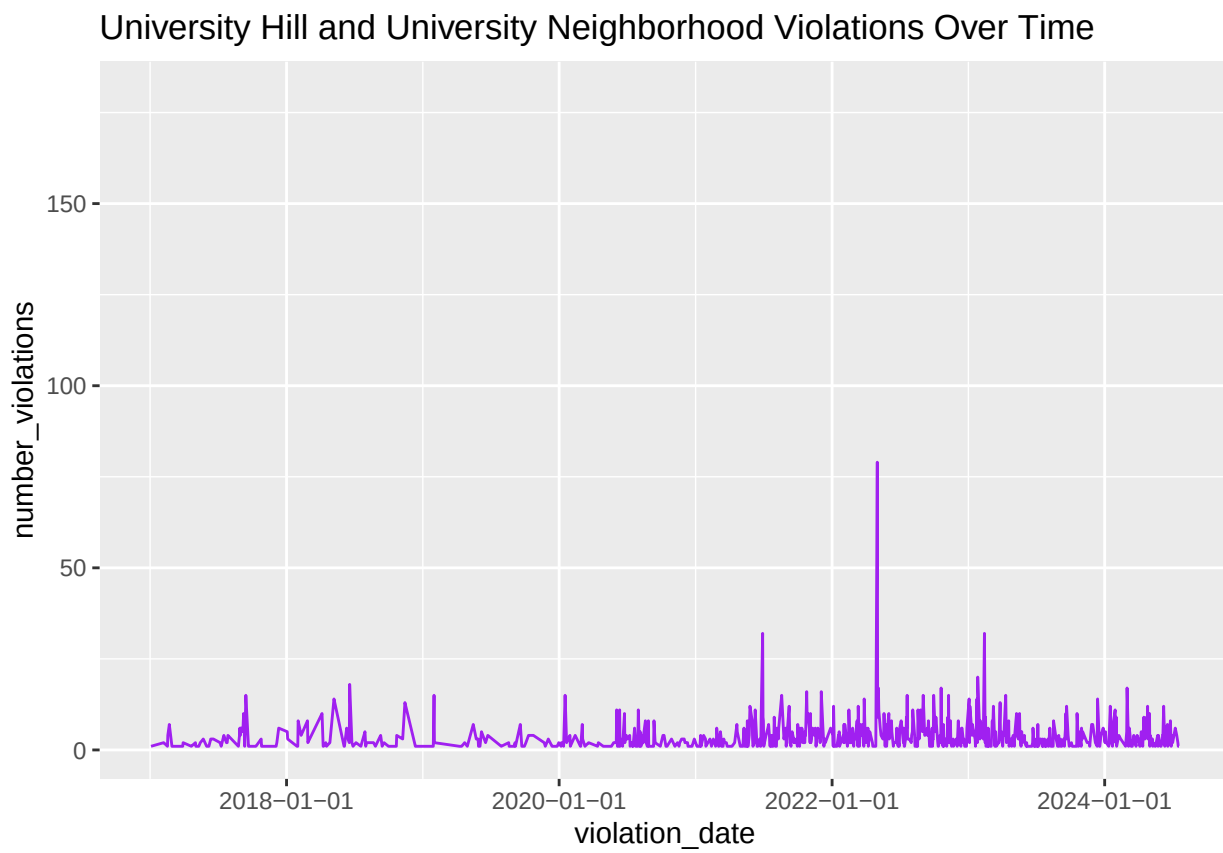
```
westcottPlot <- ggplot(westcott_by_date, aes(x = violation_date)) + geom_line(aes(y = number_violations)
    colour = "blue") + ylim(NA, 180) + scale_x_date(date_breaks = "2 years") + ggtitle("Westcott Violati
westcottPlot
```




```
downtownPlot <- ggplot(downtown_by_date, aes(x = violation_date)) + geom_line(aes(y = number_violations)
    colour = "red") + ylim(NA, 180) + scale_x_date(date_breaks = "2 years") + ggtitle("Downtown Violati
downtownPlot
```




```
universityPlot <- ggplot(university_by_date, aes(x = violation_date)) + geom_line(aes(y = number_violati
    colour = "purple") + ylim(NA, 180) + scale_x_date(date_breaks = "2 years") +
    ggtitle("University Hill and University Neighborhood Violations Over Time")
universityPlot
```

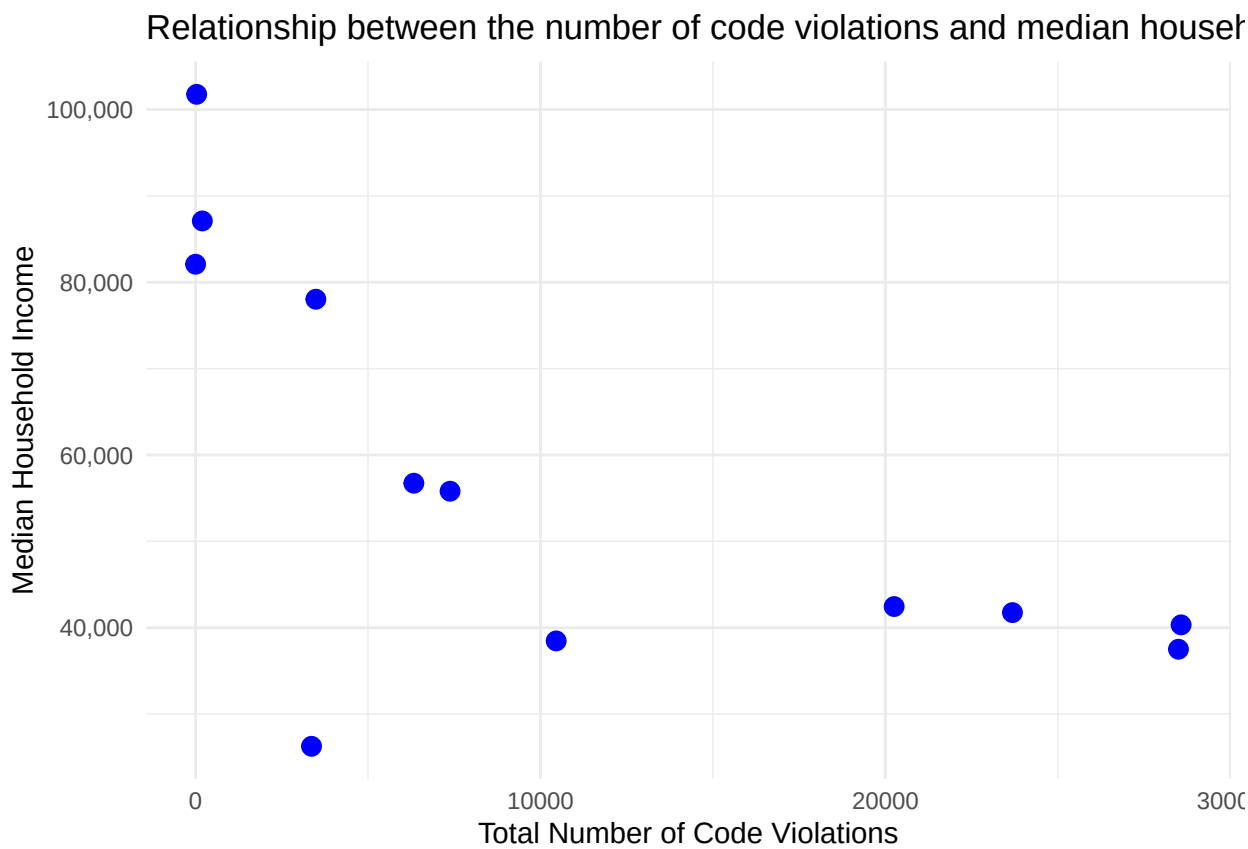


C. Now, let's create a scatter plot using violations_df to explore the relationship between number_violations and zip_median_income. This code calculates the total number of violations for each zip code and visualizes the relationship. Run the code and add comments to explain what each line does.

```
agg_data <- aggregate(violations_df$number_violations,
                      by = list(complaint_zip = violations_df$complaint_zip,
                                zip_median_income = violations_df$zip_median_income),
                      FUN = sum)
#creates dataframe from subset of violations_df retreiving complaint_zip and zip_median_income
#FUN applies sum function to number_violations

colnames(agg_data) <- c("complaint_zip", "median_income", "total_violations")
#names columns in dataframe


ggplot(agg_data, aes(x = total_violations, y = median_income)) +
  #names dataset with x and y values of the scatterplot
  geom_point(color = "blue", size = 3) + #sets color and size of points
  ggtitle("Relationship between the number of code violations and median household income in Syracuse, N
  xlab("Total Number of Code Violations") + #labels x-axis
  ylab("Median Household Income") + #labels y-axis
  scale_y_continuous(labels = function(x) format(x, big.mark = ",", scientific = FALSE)) +
  theme_minimal() #adjusts format of y-axis output for cleaner printing
```



Step 5: Executive summary

A. In 3-4 sentences, summarize the results from steps 3 and 4 - based on your interpretation of the data, do you see any patterns within and between the line charts and scatterplot? What story do these charts tell?

```
# The line plots show how the data changes over time, each neighborhood stays
# mostly consistent with random spikes on certain years. The scatterplot shows
# a negative correlation with median household income and number of code
# violations. The line plot doesn't include enough data to explain why the
# number of violations change over time, but the scatterplot reveals an
# important relationship that can be used to predict why certain neighborhoods
# have a greater amount of code violations.
```